

# Exercise 5: Task Management System

## 1. Understand Linked Lists

Singly Linked List : Each node stores data and a reference to the next node.

Doubly Linked List : Each node stores data, next pointer, and previous pointer.

| <b><i>Feature</i></b> | <b><i>Singly Linked List</i></b> | <b><i>Doubly Linked List</i></b> |
|-----------------------|----------------------------------|----------------------------------|
| Memory Usage          | Less                             | More                             |
| Forward Traversal     | Yes                              | Yes                              |
| Backward Traversal    | No                               | Yes                              |
| Deletion Efficiency   | Moderate                         | Better                           |

## 2. Setup

```
class Task {
    int taskId;
    String taskName;
    String status;
    Task next;

    Task(int taskId, String taskName, String status) {
        this.taskId = taskId;
        this.taskName = taskName;
        this.status = status;
        this.next = null;
    }
}
```

```
public String toString() {
    return "Task ID: " + taskId + " Name: " + taskName + " Status: " + status;
}
}
```

## 3. Implementation

```
class Operations {
    Task head;
    public void add(Task newTask) {
        if (head == null) {
            head = newTask;
            return;
        }
    }
}
```

```
Task temp = head;
while(temp.next != null) temp = temp.next;
temp.next = new Task;
}
```

```
public void traverse() {
    Task temp = head;
    while (temp != null) {
        System.out.println(temp);
        temp = temp.next;
    }
}
```

```
public void delete(int targetId) {
    if (head == null) {
        System.out.println("Empty List");
        return;
    }
}
```

```
if (head.taskId == targetId) {
    head = head.next;
    System.out.println("Task Deleted");
    return;
}
Task temp = head;
while(temp.next != null && temp.next.taskId != targetId) temp = temp.next;
if (temp.next == null) {
    System.out.println("Task Not Found");
    return;
}
temp.next = temp.next.next;
System.out.println("Target Deleted");
}
```

```
public class TaskManagementSystem {
    public static void main(String[] args) {
        Operations o = new Operations();

        o.add(new Task(101, "Design UI", "Pending"));
        o.add(new Task(102, "Write Code", "In Progress"));
        o.add(new Task(103, "Testing", "Pending"));
        o.add(new Task(104, "Deployment", "Not Started"));
    }
}
```

```
System.out.println("Tasks in List:");
o.traverse();
```

```
System.out.println("\nDeleting Task 103:");
o.delete(103);
```

```
System.out.println("\nTasks after Deletion:");  
o.traverse();  
}  
}
```

## 4. Analysis

| <i>Operation</i> | <i>Time Complexity</i> |
|------------------|------------------------|
| Add Task         | $O(n)$                 |
| Search Task      | $O(n)$                 |
| Traverse Task    | $O(n)$                 |
| Delete Task      | $O(n)$                 |

### Advantages of Linked Lists over Arrays

- Dynamic size.
- Efficient insertion and deletion.
- No need for contiguous memory allocation.
- Better suited for frequently changing data.